

UNIVERSITÀ DEGLI STUDI DI SALERNO



Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica applicata

Project Work - Algoritmi e Protocolli per la Sicurezza a.a. 2025/2026

Progettazione della sicurezza di un sistema di elezioni elettronico

Gruppo 26

Pisacane Francesco IE22700092

Tedesco Alessandro IE22700147

Introduzione.....	2
Capitolo 1: Modello.....	3
1.1 Attori onesti.....	3
1.2 Attaccanti (Threat Model).....	4
1.3 Proprietà di sicurezza e Analisi dei compromessi.....	9
1.3.1 Proprietà di sicurezza e priorità.....	9
1.3.2 Analisi compromessi.....	10
Capitolo 2: Soluzione.....	11
2.1 Distribuzione delle Chiavi.....	11
2.1.1 Chiavi degli Elettori.....	11
2.1.2 Chiavi dell'AE.....	13
2.2 Autenticazione.....	15
2.3 Votazione.....	17
2.4 Spoglio voti.....	19
2.5 Verifica voto.....	21
2.5.1 Voto individuale.....	21
2.5.2 Verifica universale.....	23
Capitolo 3: Analisi della sicurezza.....	25
3.1 Struttura del Token di Attivazione.....	25
3.2 Mitigazione Minacce e Preservazione Proprietà.....	26
3.3 Analisi dei Compromessi.....	28
3.4 Considerazioni sulle Modifiche Future.....	29
Capitolo 4: Implementazione.....	30
4.1 Ambiente Simulato e Scelte Tecnologiche.....	30
4.2 Analisi delle Prestazioni.....	30
4.3 Parti mancanti rispetto alla soluzione.....	32

Introduzione

Il sistema elettorale elettronico descritto nel presente documento supporta il voto a lista chiusa, escludendo la possibilità di modificare o sovrascrivere la preferenza una volta che la scheda è stata trasmessa e memorizzata.

Le scelte progettuali sono frutto delle seguenti motivazioni :

Voto a lista chiusa

Il voto a lista chiusa è direttamente implementabile sia con un voto di tipo binario (es. referendum Si/No), sia con elezioni nominali più numerose, dove ogni singolo candidato è rappresentato con un'opzione univoca.

Allo stesso tempo si vincola l'input dell'utente a delle opzioni predefinite, escludendo il testo libero, azzerando alla radice il rischio di schede nulle o contenenti nomi erroneamente scritti, semplificando il processo di scrutinio.

Esclusione modificabilità del voto

Dare la possibilità all'elettore di modificare il proprio voto può causare innumerevoli disagi al sistema, a partire dall'espansione della superficie di attacco: la necessità di gestire riscritture multiple costringerebbe il sistema a ripetere i cicli di autenticazione per lo stesso utente, moltiplicando le possibilità di attacco.

Tracciare la cronologia delle variazioni dei voti per garantire l'unicità e l'integrità dei voti introdurrebbe un overhead computazionale molto grande. Inoltre, sul piano logico e della consistenza dei dati, la sovrascrittura continua delle schede esporrebbe il database al rischio di mancati aggiornamenti o disallineamenti nel conteggio totale.

Capitolo 1: Modello

Nel primo capitolo vengono descritti gli attori onesti che interagiscono con il sistema e i loro obiettivi, gli attaccanti che ne vogliono compromettere il regolare funzionamento, e le proprietà da preservare in caso di attacchi.

1.1 Attori onesti

Di seguito vengono elencati gli attori onesti e le loro interazioni regolamentate dal sistema:

- **Autorità Elettorale (AE):** è l'ente (nazionale, regionale, aziendale, accademico...) responsabile della configurazione dell'elezione, della raccolta dei voti cifrati e del successivo scrutinio finale. Il suo obiettivo primario è garantire l'integrità delle urne digitali e l'esatta corrispondenza tra i voti espressi e quelli conteggiati, impedendo qualsiasi manipolazione dei dati memorizzati;
- **Autorità di Registrazione (AR):** è l'ente fidato che gestisce le identità e la registrazione. Controlla che un utente possa partecipare all'elezione e che riceva una sola credenziale di voto. L'AR si limita a certificare la validità dell'elettore e ad aggiornare la lista elettorale, ma non partecipa alla successiva fase di votazione, aumentando la segretezza;
- **Gestore del Server :** Rappresenta l'infrastruttura di rete tra l'Autorità Elettorale ed i client degli Elettori. Ha il compito di garantire l'alta disponibilità del servizio;
- **Elettore :** È l'utente finale del sistema che, una volta autenticato ed autorizzato, può esprimere la propria preferenza di voto (massimo 1 preferenza);
- **Certification Authority (CA) :** Entità di terze parti fidate. Il suo compito è emettere, convalidare e revocare i certificati, garantendo l'autenticità delle chiavi pubbliche
- **Custodi delle chiavi AE :** Gruppo di amministratori fidati tra i quali viene distribuita la chiave privata dell'Autorità Elettorale (utilizzando schemi di cifratura a soglia o di divisione del segreto, come lo *Shamir's Secret Sharing*), la chiave di decifratura dell'urna può essere ricomposta solo durante la fase di scrutinio finale.

1.2 Attaccanti (Threat Model)

I potenziali attaccanti del sistema sono stati suddivisi in diverse categorie, ciascuna caratterizzata da una specifica disponibilità di risorse (computazionali, finanziarie e tecnologiche), una collocazione logica o fisica all'interno del sistema e gli attacchi attesi.

- **Gestore server disonesto**

- **Descrizione e Risorse:** Attore interno dotato di medio-alta potenza di calcolo. Dispone di un accesso privilegiato alla rete del sistema e di capacità di analisi del traffico in tempo reale. Sebbene si assuma di base la sua onestà, il modello di minaccia deve prevederne la compromissione.
- **Collocazione nel Sistema:** Si posiziona al centro del canale di comunicazione tra gli Elettori e l'Autorità Elettorale (AE), controllando direttamente i nodi di routing applicativo e i server di front-end.
- **Attacchi Attesi:**
 - *timing attack:* Analisi statistica dei tempi di transito dei pacchetti per correlare l'identità dell'utente (dedotta dall'interazione iniziale con l'AR) al payload cifrato del voto inviato all'AE.
 - *Intercettazione dei Metadati:* Raccolta di indirizzi IP, timestamp e dimensioni dei messaggi nel tentativo di mappare i grafi di comunicazione e violare l'anonimato.

- **Personale negligente**

- **Descrizione e Risorse:** Minaccia passiva interna caratterizzata da bassa potenza informatica autonoma, ma con un impatto potenziale distruttivo derivante dall'accesso logico o fisico autorizzato di cui già gode. Agisce come un cavallo di Troia involontario.
- **Collocazione nel Sistema:** Endpoint interni, console di amministrazione e postazioni di lavoro del personale presso

l'Autorità di Registrazione (AR), l'Autorità Elettorale (AE) o la sala server del Gestore.

- **Attacchi Attesi:**
 - Social Engineering e Spear-Phishing: Cessione involontaria di credenziali amministrative tramite l'apertura di allegati o link malevoli.
 - data leakage Involontaria: Esposizione accidentale di parti di codice, log applicativi contenenti token o configurazioni di rete su repository pubblici.
- **Elettore disonesto:**
 - **Descrizione e Risorse:** Attore attivo esterno con risorse limitate al proprio terminale, alla propria connessione internet e all'utilizzo di script di automazione basilari. Il suo scopo è aggirare le regole della lista chiusa per un vantaggio personale o politico.
 - **Collocazione nel Sistema:** Agisce sul perimetro esterno della rete (lato Client), interfacciandosi con i messaggi di richiesta verso l'AR (registrazione) e verso l'AE tramite il Gestore (voto).
 - **Attacchi Attesi:**
 - Replay Attack: Intercettazione e successivo rinvio di un token di autorizzazione legittimo precedentemente rilasciato dall'AR, nel tentativo di esprimere voti multipli o registrare più schede.

- **Coercitore:**

- **Descrizione e Risorse:** Minaccia attiva esterna (sociale). Dispone di una bassa potenza a livello informatico, ma di risorse estremamente alte a livello sociale e fisico (capacità di intimidazione locale, coercizione psicologica o disponibilità economica).
- **Collocazione nel Sistema:** Inserito nello spazio fisico o logico privato dell'Elettore legittimo (vittima), intervenendo nel momento esatto in cui il client compila e cifra il messaggio di voto.
- **Attacchi Attesi:**
 - *Attacco Ricevuta-Dipendente:* Sfruttamento di log di conferma o ricevute crittografiche salvate in locale sul dispositivo dell'elettore per verificare post l'effettiva scelta espressa.

- **Ladro d'identità:**

- **Descrizione e Risorse:** Attore attivo esterno con medio-alta potenza. È in grado di impiegare botnet, database di credenziali compromesse e campagne mirate di spear-phishing per sottrarre chiavi e credenziali private.
- **Collocazione nel Sistema:** Interposto sul canale di comunicazione tra l'Elettore e l'AR, oppure annidato come spyware/malware direttamente sul dispositivo dell'Elettore.
- **Attacchi Attesi:**
 - *Dictionary e brute-force attacks* : sfruttando le credenziali sottratte potrebbe richiedere il token di voto all'AR e consegnare la preferenza all'AE prima dell'utente reale, causandone il rifiuto del voto per duplicazione.

- **Ransomware e RaaS (Ransomware as a Service):**
 - **Descrizione e Risorse:** Gruppi criminali organizzati esterni motivati dal solo profitto economico. Dispongono di risorse alte: budget elevato, accesso a vulnerabilità zero-day e infrastrutture distribuite per garantire il proprio anonimato.
 - **Collocazione nel Sistema:** Penetra all'interno delle reti locali e dei server di backend dell'Autorità Elettorale (AE) o del database centrale gestito dall'AR, dopo aver superato i firewall perimetrali.
 - **Attacchi Attesi:**
 - *Cifratura dell'Infrastruttura:* Blocco dei database e dei server d'urna nel bel mezzo della sessione elettorale, interrompendo lo scambio di messaggi e impedendo il calcolo.

- **Cyber-Warfare (State-Sponsored):**
 - **Descrizione e Risorse:** Attori governativi esteri con risorse quasi illimitate: potenza di calcolo su scala nazionale, team di hacker d'élite a tempo pieno, strumenti crittografici e cyber-armi avanzate. L'obiettivo è il sabotaggio geopolitico e il minare la fiducia nella democrazia.
 - **Collocazione nel Sistema:** Può posizionarsi sui nodi di routing dei carrier nazionali (ISP), intercettare le Certification Authority (CA) o agire come Advanced Persistent Threat (APT) silente nei sistemi centrali dell'AE.
 - **Attacchi Attesi:**
 - *Criptoanalisi Avanzata/chosen-plaintext attacks :* Tentativi di decifrare le sessioni TLS o le firme cieche del sistema sfruttando zero-day crittografici o analizzando le emissioni dei server (attacchi side-channel).
 - *Compromissione dei Custodi delle Chiavi:* Attacchi mirati ai singoli custodi per sottrarre e ricomporre in

anticipo le quote della chiave privata dell'AE necessaria alla decifratura dei voti.

- **Hacktivisti:**

- **Descrizione e Risorse:** Gruppi mossi da motivazioni ideologiche. Possiedono una media potenza, basata su botnet, tool open-source ed efficienti dinamiche di coordinamento di massa online per saturare i sistemi.
- **Collocazione nel Sistema:** Interamente esterno all'infrastruttura; colpisce i canali di comunicazione pubblici che portano ai server di front-end del Gestore del Server e dell'AR.
- **Attacchi Attesi:**
 - *Distributed Denial of Service (DDoS):* Inondazione di richieste (es. HTTP/HTTPS Flood o SYN Flood) per esaurire la banda del Gestore, impedendo agli elettori di stabilire la connessione e inviare i messaggi di voto prima della scadenza

1.3 Proprietà di sicurezza e Analisi dei compromessi

Anche in presenza di attacchi, il sistema deve garantire le proprietà fondamentali quali autenticità, unicità, segretezza, integrità e verificabilità (individuale e universale). Ognuno di queste è fondamentale per garantire riservatezza e trasparenza allo stesso tempo, tuttavia c'è bisogno di attuare dei compromessi tra queste proprietà di base in conflitto tra di loro per trovare un trade off ottimale.

1.3.1 Proprietà di sicurezza e priorità

Massima priorità : *Integrità, Autenticità, Unicità.*

Rappresentano le fondamenta del sistema. In caso di compromissione di uno di questi pilastri l'intera votazione perde validità legale e matematica, rendendo superflua qualsiasi altra garanzia.

Il sistema deve garantire che l'accesso alle urne sia riservato esclusivamente agli utenti registrati dall'Autorità di Registrazione (AR) e deve evitare la possibilità di esprimere più voti per lo stesso elettore. I dati memorizzati nell'urna digitale devono essere immutabili e resistenti ad attacchi di manipolazione

Alta Priorità: *Segretezza del Voto*

A differenza di altri sistemi informatici dove l'anonimato può subire un degrado controllato a fini di audit in caso di incidente, nel voto elettronico la segretezza deve rimanere assoluta e non sacrificabile.

Il sistema deve impedire la correlazione tra l'identità anagrafica dell'elettore e la preferenza espressa. Il protocollo deve adottare schemi di cifratura probabilistica con garanzia di indistinguibilità sotto attacco a testo in chiaro

Media Priorità: *Verificabilità Universale e Verificabilità Individuale*

La verificabilità universale permette a chiunque di verificare, tramite computazioni pubbliche, che la somma finale corrisponda esattamente alla totalità dei voti validi registrati, sebbene sia vitale per dimostrare l'onestà dello scrutinio, in caso di attacco può

essere ritardata nel tempo e recuperata una volta ripristinata la rete.

La verificabilità individuale permette all'elettore di generare una prova crittografica che attesti che il proprio voto sia stato correttamente incluso nello scrutinio finale, può essere momentaneamente sospesa in caso di attacchi, senza compromettere l'integrità del voto.

1.3.2 Analisi compromessi

Segretezza del Voto vs Verificabilità Individuale: È il compromesso più critico. Se il protocollo fornisse all'elettore una ricevuta contenente la prova matematica della propria preferenza, un *Coercitore* potrebbe costringerlo a esibirla.

Il protocollo deve implementare per l'elettore la possibilità di verificare che il proprio *ciphertext* sia presente nell'urna, ma non deve disporre di alcun dato trasferibile a terzi capace di rivelare il *voto* associato.

Autenticazione vs Pseudo Anonimato: Per impedire il *Double Voting*, il sistema deve identificare univocamente l'elettore; per preservare lo pseudo anonimato, l'urna non deve conoscere tale identità. Per risolvere questo conflitto il protocollo andrà a dividere le responsabilità in due enti diversi (AR ed AE), i quali gestiranno l'autenticazione ed il voto in modo separato, così da non compromettere l'identità dell'elettore

Robustezza Crittografica vs Efficienza Computazionale:

L'adozione di strategie crittografiche avanzate per prevenire possibili attacchi aumenta l'overhead computazionale e il consumo di banda. Questo trade-off è fondamentale per gestire l'enorme mole di elettori che si potrebbero presentare alle votazioni e per non creare disservizi.

Capitolo 2: Soluzione

In questo capitolo ci si appropria al modello descritto nel capitolo precedente, progettando una possibile soluzione che possa implementare il sistema descrivendo tutte le interazioni degli attori onesti.

2.1 Distribuzione delle Chiavi

Un requisito fondamentale per garantire l'integrità, l'autenticità e la segretezza del sistema è una robusta gestione delle chiavi crittografiche. Il sistema si basa su un'Infrastruttura a Chiave Pubblica esistente, gestita a livello nazionale.

2.1.1 Chiavi degli Elettori

Per garantire la massima segretezza e prevenire frodi interne, il sistema non prevede la generazione o la consegna diretta delle chiavi da parte di un'entità centrale. Nessun server conoscerà mai la chiave privata dell'elettore SK_E . Si adotta invece un meccanismo di verifica dell'identità tramite un canale Out-of-Band. L'AR genera un token di attivazione monouso per ogni avente diritto, recapitandolo tramite un canale sicuro, generalmente via postale presso il recapito dell'elettore, ma nel caso in cui l'elezione sia interna a un'organizzazione (università, azienda...) si può valutare l'invio del token tramite la posta elettronica di quella organizzazione. L'AR salva i token di autenticazione sul server, cifrati tramite hash, associati ognuno al codice fiscale dell'elettore.

L'elettore, tramite l'app, genera localmente la propria coppia di chiavi asimmetriche PK_E e SK_E . Successivamente, utilizzando il token ricevuto e il suo codice fiscale, si autentica sul portale dell'RA per richiedere la certificazione. L'AR, verificata la validità del token, firma digitalmente la chiave pubblica dell'elettore PK_E , emettendo il certificato di voto $CERT_E = \text{SIGN}_{SK-AR} \{ID_{AR}, PK_E\}$, con l' ID_{AR} calcolato cifrando il codice fiscale dell'elettore tramite hash con aggiunta di salt, e successivamente comunicato all'AE, tramite canale Out-Of-Band. L'AE cifra l' ID_{AR} ricevuto tramite hash con aggiunta di pepper, ossia un salt che però non è casuale ma conosciuto e ben custodito dall'AE, e infine salva l' ID_{pepper} nella lista elettorale. L'ulteriore cifratura con aggiunta di pepper è necessaria per prevenire la segretezza in caso di attacchi all'AR.

Scambio dei Messaggi

Entità coinvolte:

- E: Elettore
- AR: Autorità di Registrazione
- AE: Autorità Elettorale

L'AR genera il token e lo recapita in modo sicuro tramite canale Out-Of-Band.

- $AR \rightarrow E: \{Token\}$

L'elettore genera localmente PK_E e SK_E , poi contatta l'AR per l'autenticazione.

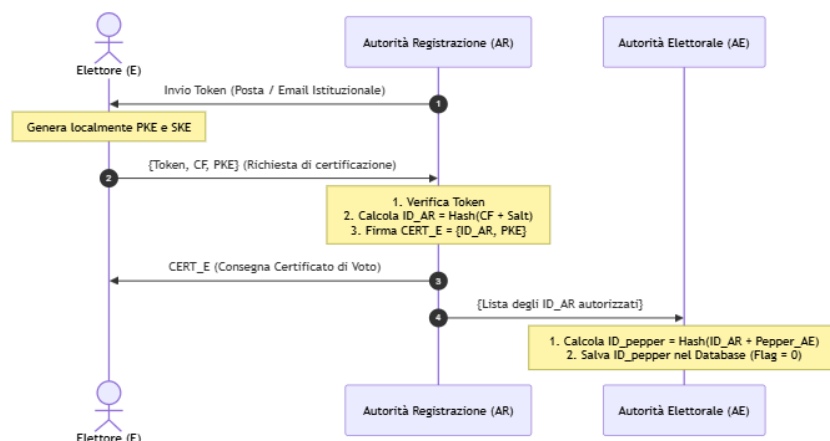
- $E \rightarrow AR: \{Token, CF, PK_E\}$

L'AR verifica il token, calcola l' ID_{AR} e firma il certificato.

- Calcolo interno AR: $ID_{AR} = H(CF \parallel Salt)$
- Firma AR: $CERT_E = \text{Sign}_{SK-AR}(ID_{AR}, PK_E)$
- $AR \rightarrow E: CERT_E$

L'AR comunica all'AE chi è stato autorizzato al voto, senza rivelare i nomi veri. L'AE blinda i dati ricevuti prima di salvarli.

- $AR \rightarrow AE: \{Lista ID_{AR}\}$
- Calcolo interno AE: $ID_{pepper} = H(ID_{AR} \parallel Pepper_{AE})$
- AE salva ID_{pepper} nella Lista Elettorale con Flag voto impostato a 0.



2.1.2 Chiavi dell'AE

L'Autorità Elettorale genera due coppie di chiavi asimmetriche PK_{AE} , SK_{AE} per ogni elezione, una serve a cifrare e decifrare i voti, l'altra serve a firmare le ricevute dei voti. Per la prima coppia:

- La Chiave Pubblica PK_{AE} viene certificata da una CA fidata e pubblicata su canali ufficiali (es. sito istituzionale) prima dell'inizio delle votazioni. Questa chiave viene utilizzata dai client degli elettori per cifrare le schede di voto tramite RSA-OAEP, garantendo che solo l'AE possa decifrarle.
- La Chiave Privata SK_{AE} è il segreto più critico del sistema. Per evitare un single point of failure, la SK_{AE} non viene mai memorizzata in chiaro su un singolo server. Viene suddivisa fin dalla generazione utilizzando uno schema di condivisione di segreti. I frammenti della chiave vengono distribuiti in modo sicuro, tramite canali out-of-band o protocolli di scambio chiavi autenticati, a N custodi indipendenti (enti di fiducia). Solo alla chiusura delle urne, un numero minimo T di custodi dovrà collaborare per permettere la fase di decifratura.

Scambio dei Messaggi

Entità coinvolte:

- AE: Autorità Elettorale
- CA: Certification Authority Fidata
- C_i : Custodi indipendenti ($C_1, C_2, \dots, C_N$)

L'AE genera le chiavi e richiede al CA di certificarne l'autenticità.

- Operazione interna AE: Genera la coppia di chiavi (PK_{AE}, SK_{AE}) .
- $AE \rightarrow CA$: $\{ReqCert, PK_{AE}\}$
- Operazione interna CA: Verifica la richiesta e firma la chiave pubblica dell'AE.
- $CA \rightarrow AE$: $CERT_{AE} = Sign_{SK-CA}(PK_{AE})$
- Pubblicazione AE: Pubblica $CERT_{AE}$ affinché i client degli elettori possano scaricarlo per usare RSA-OAEP.

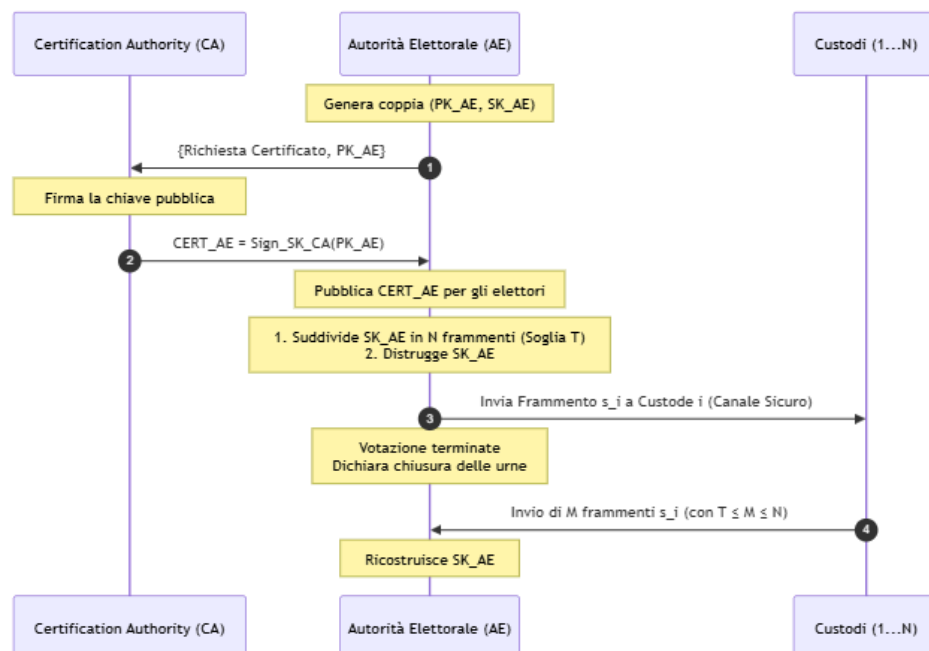
Per evitare il single point of failure, l'AE frammenta la chiave privata e la distribuisce.

- Operazione interna AE: Applica uno schema di condivisione del segreto dividendo la SK_{AE} in N frammenti ($s_1, s_2, \dots, s_N$), definendo una soglia minima T .
- $AE \rightarrow C_i: \{s_i\}$ tramite canale sicuro out-of-band
- Operazione critica AE: Distrugge la SK_{AE} intera.

Fase 3: Ricostruzione del Segreto (Post-elezione)

A urne chiuse, per iniziare lo scrutinio, l'AE deve recuperare il segreto collaborando con i custodi.

- Operazione interna AE: Dichiarare la chiusura formale delle urne e apre il canale di ricomposizione.
- $C_1, C_2, \dots, C_M$ con $T \leq M \leq N \rightarrow AE$: Inviano i propri frammenti $\{s_i\}$ ad AE per la decifratura.
- Operazione interna AE: Unisce gli M frammenti ricevuti per calcolare la chiave originale: $SK_{AE} = \text{Reconstruct}(s_1, s_2, \dots, s_M)$.



2.2 Autenticazione

Il primo passo per avere un'elezione sicura è autenticare gli elettori. L'Autorità Elettorale dispone della lista elettorale contenente tutti gli utenti autorizzati a votare sotto forma di identificativi pseudonimi ID_{pepper} , ognuno con il proprio flag 0-1 che indica se l'utente ha già espresso la preferenza di voto.

L'utente richiede l'accesso presentando all'AE il proprio certificato di voto $CERT_E$ rilasciato dall'AR. Il sistema dell'AE estrae l' ID_{AR} contenuto nel certificato, ne calcola l'hash con l'aggiunta del proprio pepper e lo cerca nella lista. Se l'identificativo è presente e il flag indica che non ha ancora votato, l'AE avvia un protocollo di Challenge-Response.

L'AE genera un Nonce e lo invia come challenge all'elettore.

L'applicazione dell'elettore firma il Nonce con la propria chiave privata SK_E e lo restituisce al server.

Solo dopo aver verificato questa firma tramite la chiave pubblica contenuta nel certificato, l'AE ha la certezza dell'identità dell'utente, aggiorna il flag a 1 e lo autorizza a inviare la scheda elettorale.

Scambio dei Messaggi

Entità coinvolte:

- E: Elettore
- AE: Autorità Elettorale

L'elettore richiede di autenticarsi e presenta il certificato di voto all'AE

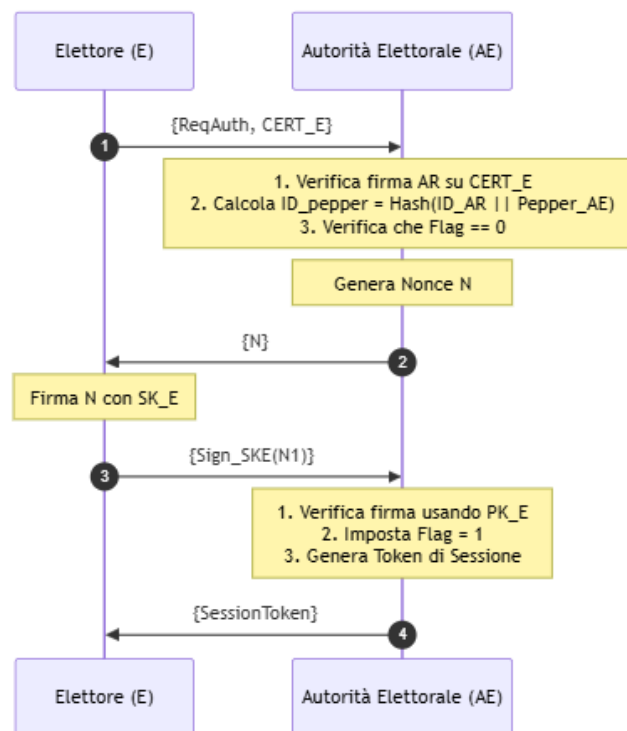
- $E \rightarrow AE: \{ReqAuth, CERT_E\}$
- Operazioni interne AE:
 1. Verifica la firma dell'AR sul $CERT_E$.
 2. Estrae ID_{AR} e PK_E .
 3. Calcola $ID_{\text{pepper}} = H(ID_{AR} || Pepper_{AE})$.
 4. Trova ID_{pepper} nella lista e verifica che il Flag sia 0.

AE avvia il protocollo di Challenge-Response

- Operazione interna AE: Genera un Nonce N.
- $AE \rightarrow E: \{N\}$

E firma la challenge poi AE la verifica, autorizzando E

- Operazione interna E: Firma la challenge con la propria chiave privata $\text{Sign}_{\text{SK-E}}(N)$.
- $E \rightarrow AE: \{\text{Sign}_{\text{SK-E}}(N)\}$
- Operazioni interne AE:
 1. Verifica la firma sul Nonce utilizzando la PK_E dell'utente.
 2. Imposta il Flag a 1 nella Lista Elettorale.
 3. Genera un token di sessione per abilitare il caricamento del voto.
- $AE \rightarrow E: \{\text{SessionToken}\}$



2.3 Votazione

Una volta autenticato, l'elettore ha la possibilità di esprimere la sua preferenza di voto.

La preferenza viene espressa tramite una lista chiusa di opzioni (candidati, partiti, Sì/No...). La lista delle opzioni viene rappresentata come un array di bit (messi a 0 di default) con lunghezza pari al numero di opzioni disponibili. Quando l'elettore seleziona un'opzione, il bit corrispondente viene messo a 1. Se un elettore si astiene, la sua scheda sarà un array di zeri ma il flag sarà comunque messo ad 1, indicando la votazione effettuata.

Per aumentare la sicurezza, il sistema va a crittografare l'array del voto insieme al timestamp, per cifrare la scheda, si userà un algoritmo probabilistico come RSA-OAEP, in questo modo il voto è protetto da attacchi chosen-plaintext (CPA) e due voti uguali avranno due ciphertext diversi. I voti vengono infine memorizzati in un Merkle Tree che garantisce l'integrità dei voti e il dispositivo dell'utente memorizza una ricevuta contenente l'hash del proprio voto.

Il voto viene cifrato con la PK_{AE} appartenente alla prima coppia di chiavi (quella contenente la SK_{AE} distrutta), mentre la ricevuta viene firmata usando la $SK_{AE-SIGN}$ appartenente ad una seconda coppia di chiavi dedicata alla firma delle ricevute.

Scambio dei messaggi

Entità coinvolte:

- E: elettore
- AE: Autorità Elettorale

E genera il crittogramma del voto

- Operazione interna E: $C_v = \text{Enc}_{\text{PK-AE}}(v \parallel \text{padding} \parallel \text{Timestamp})$

E invia all'AE il suo Session Token e il voto cifrato

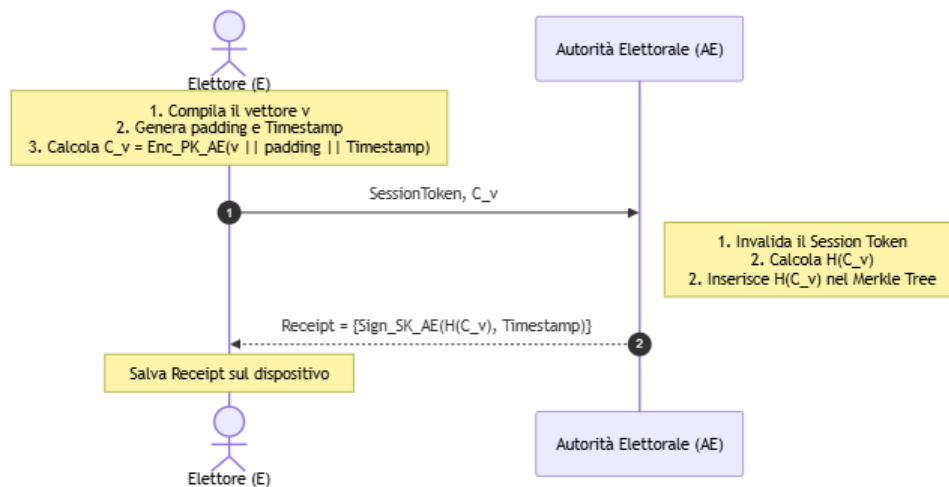
- $E \rightarrow AE: \{\text{SessionToken}, C_v\}$

AE elabora il voto ricevuto:

- Operazione interna AE:
 - Invalida il SessionToken.
 - Calcola l'hash del crittogramma $H(C_v)$ e lo inserisce come foglia nel Merkle Tree dell'elezione.

AE invia ad E una ricevuta che userà alla pubblicazione dei voti per consultare il proprio voto.

- $AE \rightarrow E: \text{Receipt} = \{\text{Sign}_{\text{SK-AE-SIGN}}(H(C_v), \text{Timestamp})\}$



2.4 Spoglio voti

Una volta terminato il periodo di votazione, l'Autorità Elettorale calcola per ogni foglia del Merkle Tree, la Merkle Proof e pubblica ufficialmente la tabella delle coppie (Merkle Proof - Hash della foglia) insieme alla Merkle Root finale, congelando di fatto l'aggiunta di nuovi voti.

Per evitare attacchi di correlazione temporale tra l'invio dei voti e il cambio del flag nella tabella dei codici fiscali, l'AE non decifra i voti nell'ordine in cui sono arrivati, ma alla chiusura delle urne, per garantire che il gestore del server o l'Autorità non possano collegare il momento del voto (il timestamp di ricezione) al crittogramma, viene applicato un processo di rimescolamento.

Successivamente il custode della chiave privata attiva il protocollo di decifratura e i vettori binari risultanti vengono contati per conoscere il numero di voti per ogni candidato.

Come anticipato nel paragrafo 2.1, se la chiave privata dell'Autorità Elettorale è singola, il custode della chiave sarà un single failure point, e potrebbe essere soggetto di molteplici attacchi, oltre a rappresentare una parte di cui fidarsi. Per sopperire a ciò si divide la chiave in più parti, da affidare ognuna a un proprio custode, ed eventualmente permettere il funzionamento anche senza qualcuna di esse (Threshold Scheme).

Scambio dei Messaggi

Entità coinvolte:

- AE: Autorità Elettorale
- C_i : Custodi indipendenti ($C_1, C_2, \dots, C_N$)

L'AE chiude le votazioni e rende pubblici i dati crittografati per permettere agli elettori di verificare che il loro voto sia nel Merkle Tree.

- Operazione interna AE:
 - Calcola la MerkleRoot e tutte le MerkleProof_i.
 - Pubblica la MerkleRoot e la lista con i voti cifrati, ognuno accoppiato alla propria MerkleProof

Operazione locale sul server dell'AE per distruggere la correlazione temporale.

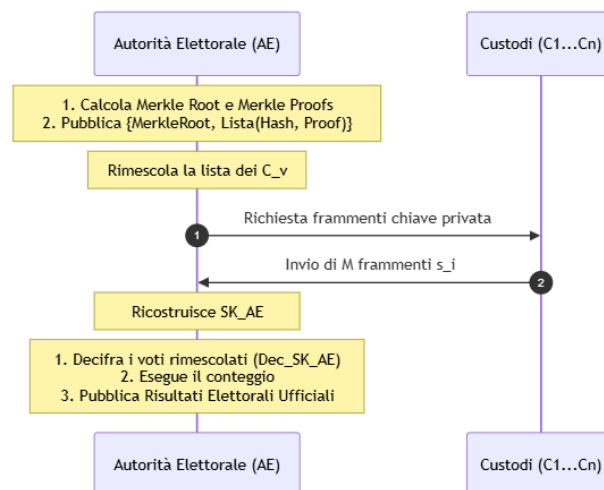
- Operazione interna AE: Inserisce tutti i crittogrammi C_v in una lista ed esegue il rimescolamento. Ottiene una nuova lista di crittogrammi anonimizzati $C'_v = \text{Shuffle}(C_v)$.

L'AE raccoglie i frammenti necessari per abilitare la decifratura.

- $AE \rightarrow C_i$: ReqFragment (Richiesta di invio dei frammenti della chiave a ogni custode)
- $C_1, C_2, \dots, C_M$ con $T \leq M \leq N \rightarrow AE$: $\{s_1\}, \{s_2\}, \dots, \{s_M\}$ (Invio di M frammenti)
- Operazione interna AE: $SK_{AE} = \text{Reconstruct}(s_1, s_2, \dots, s_M)$

AE decifra e conta i voti.

- Operazione interna AE: Decifra ogni crittogramma della lista rimescolata: $v = \text{Dec}_{SK_{AE}}(C'_v)$.
- Operazione finale: Somma i vettori binari v e pubblica i risultati.



2.5 Verifica voto

Attraverso meccanismi di verifica individuale e universale, il sistema permette agli elettori di confermare che il proprio voto sia stato registrato correttamente e, contemporaneamente, consente agli osservatori esterni di validare l'integrità del risultato finale, il tutto senza mai compromettere la segretezza della scelta espressa.

2.5.1 Voto individuale

Per garantire la verificabilità individuale evitando problemi di coercizione e vendita di voti, il programma implementa un sistema che si basa su ricevute di voto e si appoggia alla struttura dei merkle tree.

Alla chiusura delle urne, l'utente avrà la possibilità di verificare la presenza del proprio voto nel conteggio finale in questo modo: dopo aver recuperato l'hash del proprio voto memorizzato sul dispositivo, potrà recuperare la Merkle Proof dalla tabella e con questi 2 elementi è in grado di ricalcolare in locale il merkle root per confrontarlo con quello ufficiale pubblicato dall'Autorità elettorale.

Se i 2 valori coincidono, l'elettore ha la certezza che il proprio voto è presente nel computo finale, senza che ci siano state modifiche al proprio voto.

Scambio dei messaggi

Entità coinvolte:

- E: Elettore
- AE: Autorità Elettorale (in particolare la bacheca dei risultati)

L'elettore legge la ricevuta sul proprio telefono e chiede alla bacheca i dati per validarla.

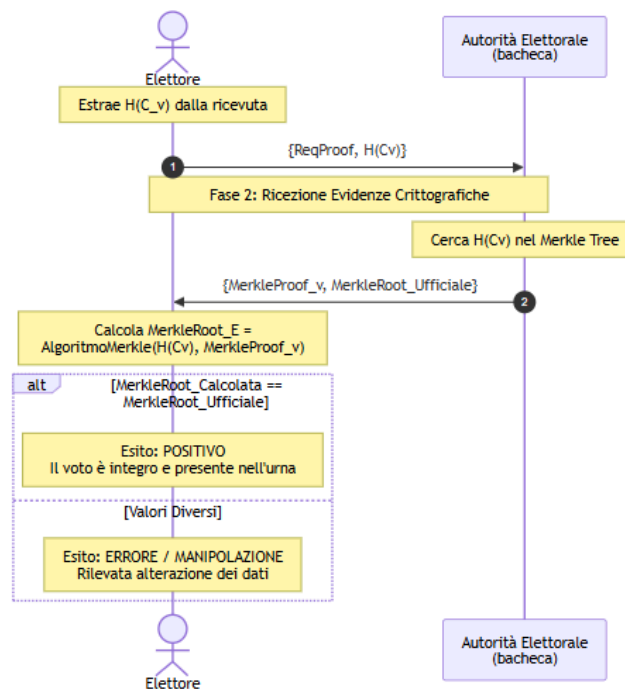
- Operazione interna E: Estrae $H(C_v)$ dalla Receipt salvata in memoria.
- $E \rightarrow AE$: $\{\text{ReqProof}, H(C_v)\}$

La bacheca restituisce la prova e la radice ufficiale dell'intera elezione.

- $AE \rightarrow E$: $\{\text{MerkleProof}_v, \text{MerkleRoot}_{AE} \text{ (pubblicata)}\}$

L'App dell'elettore esegue il calcolo in locale.

- Operazione interna E: Calcola la radice stimata combinando l'hash del voto e i nodi fratelli della prova:
 $\text{MerkleRoot}_E \text{ (calcolata)} = \text{Merkle}(H(C_v), \text{MerkleProof}_v)$
- Controllo di uguaglianza: Verifica se $\text{MerkleRoot}_E == \text{MerkleRoot}_{AE}$



2.5.2 Verifica universale

Un requisito ideale per il sistema è la verificabilità universale, ovvero la possibilità per qualsiasi cittadino di ricalcolare e verificare l'esattezza dello spoglio senza conoscere la chiave privata dell'AE.

Nel presente progetto si è scelto di utilizzare lo schema asimmetrico probabilistico RSA-OAEP. Tuttavia, un'analisi critica del protocollo evidenzia un limite strutturale di questa scelta rispetto alla verificabilità universale. Per permettere a un elettore, o comunque uno spettatore esterno, di verificare la decifratura di un voto, l'AE dovrebbe rendere pubblico non solo il voto in chiaro, ma anche il crittogramma, il timestamp e il padding casuale associati, così si potrebbero cifrare tramite la PK_{AE} e confrontare il voto ottenuto con quello cifrato pubblicato. In questo modo però, pubblicando tali dati nascerebbe una grave vulnerabilità: un attaccante (soprattutto un coercitore) potrebbe ricalcolare l'hash del crittogramma svelato e cercarlo all'interno della Merkle Proof pubblicata per la verifica individuale. Questo attacco annullerebbe di fatto la protezione offerta dal rimescolamento iniziale, ricollegando inequivocabilmente il voto in chiaro all'elettore che possiede la ricevuta, distruggendo il meccanismo della ricevuta e aprendo le porte alla coercizione.

Di conseguenza, per mantenere l'anonimato e prevenire la compravendita dei voti, si è scelto un approccio conservativo: il sistema rinuncia intenzionalmente alla verificabilità universale pubblica. Il padding originale viene distrutto e non divulgato, affidando la correttezza della decifratura esclusivamente alla distribuzione della fiducia garantita dal Threshold Scheme sui custodi della chiave privata.

Per superare questo limite senza compromettere l'anonimato, un'evoluzione futura del sistema richiederebbe l'impiego di strumenti crittografici avanzati non trattati nell'attuale implementazione. Nello specifico, si renderebbe necessario l'uso di Zero-Knowledge Proofs associate a schemi di crittografia omomorfica. Tali tecnologie permetterebbero all'AE di generare una prova matematica pubblica che attesti la corretta decifratura dei voti, verificabile da chiunque, senza tuttavia svelare i parametri di cifratura originali né alcun legame con i crittogrammi inseriti nell'urna.

Un ulteriore rischio legato alla pubblicazione dei risultati è l'attacco statistico basato sulla frequenza. Se l'Autorità Elettorale pubblicasse l'elenco dei voti decifrati uno ad uno, anche se rimescolati, in presenza di un candidato fortemente dominante (es. con il 90% delle preferenze), l'efficacia del rimescolamento verrebbe drasticamente ridotta. Un attaccante di rete potrebbe dedurre con un'altissima probabilità il voto di un cittadino semplicemente associando il pacchetto cifrato intercettato alla preferenza statisticamente dominante.

Per mitigare questo rischio e concludere il protocollo in sicurezza, il sistema si limita a mostrare sulla Bacheca Pubblica esclusivamente il conteggio aggregato finale dei voti (i totali per ciascun candidato) e non la lista disaggregata delle schede in chiaro.

In questo modo, l'elezione si conclude accettando un modello di fiducia distribuito: i cittadini verificano individualmente che il proprio crittogramma sia nell'urna tramite Merkle Proof, i custodi garantiscono l'onestà dello scrutinio vigilando sulla ricostruzione della chiave Threshold Scheme, e l'anonimato viene preservato rinunciando alla pubblicazione dei singoli voti in chiaro per impedire attacchi di correlazione temporale e statistica.

Capitolo 3: Analisi della sicurezza

L'obiettivo di questo capitolo è analizzare la sicurezza della soluzione proposta nel Capitolo 2, valutandone l'efficacia nel preservare le proprietà definite nel Capitolo 1 contro le minacce identificate, evidenziando i compromessi progettuali adottati e le assunzioni di fiducia.

3.1 Struttura del Token di Attivazione

Per mitigare gli attacchi provenienti da un ladro d'identità remoto, il sistema non permette la registrazione tramite semplici credenziali online, ma si affida a un canale di distribuzione Out-of-Band gestito dall'AR.

Il Token di Attivazione è strutturato per prevenire attacchi di brute-force e di errori di digitazione. La sua struttura logica è la seguente:

- **Codice ad Alta Entropia:** Una stringa alfanumerica di 16 caratteri.
- **Checksum integrato:** Gli ultimi caratteri del token fungono da codice di controllo. Questo permette all'app di validare formalmente il token in locale, senza dover interrogare i server dell'AR in caso di banale errore di battitura, limitando i rischi di attacchi DDoS involontari generati dagli utenti.
- **Proprietà Single-Use:** Il token è associato temporaneamente al Codice Fiscale dell'elettore nel server dell'AR. Appena l'utente lo utilizza per firmare il certificato di voto, il token viene invalidato e distrutto dal server.
- **Time To Live Elettorale:** si può valutare di attribuire al token una scadenza prestabilita. Qualora non venga utilizzato entro la chiusura delle urne, decade automaticamente.

Inoltre, la busta postale che trasporta il token non contiene al suo interno il codice fiscale dell'elettore. Chiunque riesca ad intercettare il token non potrà votare al posto dell'elettore, a meno che non conosca anche il suo codice fiscale.

3.2 Mitigazione Minacce e Preservazione Proprietà

Il design del sistema è stato strutturato per neutralizzare gli attaccanti descritti nel Threat Model, accettando compromessi precisi.

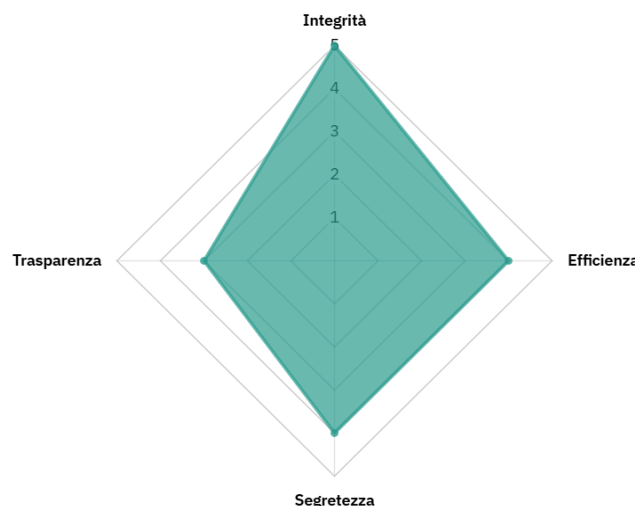
- Contro Ladro d'Identità e Elettore Disonesto (Autenticità e Unicità):
 - Soluzione: La combinazione del Token di Attivazione Out-of-Band e del certificato di voto garantisce che nessun attaccante online possa rubare il diritto di voto senza aver compromesso anche il canale fisico. La proprietà Single-Use del token impedisce all'Elettore Disonesto di richiedere chiavi multiple. L'unicità del voto è garantita dal flag 0-1 nella lista elettorale dell'AE.
 - Compromesso (Superato): Aggiornare il flag durante l'invio del voto avrebbe esposto il sistema a correlazione temporale. Si è scelto di separare le fasi: il flag viene impostato a 1 durante la fase di Challenge-Response, slegando l'identità dal momento esatto dell'inserimento della scheda nell'urna.
- Contro Gestore del Server Disonesto e Cyber-Warfare (Integrità e Correlazione Temporale):
 - Soluzione: L'integrità è assicurata dalla struttura Append-Only del Merkle Tree. Qualsiasi alterazione di un ciphertext da parte del server invaliderebbe immediatamente la Merkle Root pubblica. Per proteggere la segretezza dai timing attack, ossia ricostruire l'identità dell'elettore controllando l'ordine temporale, il protocollo impone il rimescolamento dei crittogrammi prima della decifratura.
 - Compromesso (Superato): L'introduzione di una fase di rimescolamento introduce un inevitabile ritardo operativo, impedendo il conteggio dei voti in tempo reale. Si è scelto di sacrificare l'immediatezza dello scrutinio a favore di una sicurezza inscalfibile: i voti vengono accumulati in forma cifrata e l'elaborazione avviene solo a fine sessione, neutralizzando così ogni tentativo di correlazione basato sull'ordine di arrivo dei pacchetti.

- Contro il Coercitore (Segretezza vs Trasparenza):
 - Soluzione: La coercizione è la minaccia più letale nel voto remoto. Se l'AE pubblicasse i voti in chiaro per la verificabilità universale, un coercitore potrebbe utilizzare l'hash della ricevuta in possesso dell'elettore per ricollegarlo al voto decifrato. Per neutralizzare la coercizione, il sistema impone la distruzione dei singoli voti in chiaro e la sola pubblicazione della macro-aggregazione dei risultati.
 - Compromesso: Si sacrifica consapevolmente la Verificabilità Universale del singolo voto per garantire la sicurezza dei dati in mano ai coercitori.
- Contro Ransomware, DDoS e Hacktivisti (Disponibilità vs Integrità):
 - Soluzione: In caso di attacco DDoS o cifratura dell'infrastruttura di rete, l'utilizzo del pepper segreto a protezione degli ID degli elettori previene il data leakage di massa.
 - Compromesso: A fronte di un attacco volto a distruggere la disponibilità, il sistema predilige un blocco del servizio pur di garantire l'integrità dell'urna, ossia il Merkle Tree, evitando il falsamento del risultato dell'elezione.

3.3 Analisi dei Compromessi

Per valutare il bilanciamento architettonico raggiunto, analizziamo i quattro vertici teorici del nostro sistema in base alle scelte implementative:

1. **Integrità** - Massima: L'utilizzo del Merkle Tree, in cui la radice viene firmata e resa pubblica, rende crittograficamente impraticabile l'alterazione, l'aggiunta o l'eliminazione di un voto a posteriori.
2. **Efficienza** - Alta: Rinunciando a costrutti crittografici pesanti per lo spoglio, le operazioni richieste al client e al server si limitano ad hash (SHA-256), cifratura asimmetrica (RSA-OAEP) ed elaborazione di alberi binari, risultando estremamente scalabili. Per raggiungere l'efficienza massima, il sistema dovrebbe adottare meccanismi crittografici più semplici a livello computazionale, che però impatterebbero negativamente su integrità e segretezza.
3. **Confidenzialità e Segretezza** - Alta: Garantita da RSA, dallo Shuffling e dalla Macro-Aggregazione finale, che disinnescano l'analisi delle frequenze. Per raggiungere la segretezza massima, dovremmo eliminare i custodi e mantenere la SK_{AE} intera, causando numerosi problemi dal punto di vista dell'integrità.
4. **Trasparenza** - Bilanciata: Il compromesso principale del progetto. La trasparenza è massima a livello individuale (Merkle Proof), ma risulta degradata a livello pubblico per favorire la mitigazione della coercizione. Per raggiungere la trasparenza massima si potrebbero pubblicare i voti (cifrati e in chiaro) associati al padding e al timestamp, così ogni utente può cifrarli con la PK_{AE} e verificare autonomamente che coincidano con i voti cifrati e da qui calcolare la Merkle Root, ma in questo modo la segretezza sarebbe nulla.



3.4 Considerazioni sulle Modifiche Future

Durante la progettazione del sistema, si sono analizzate possibili evoluzioni future per superare i limiti attualmente accettati:

- Da risultati aggregati alla Zero-Knowledge Proof (ZKP): Il limite principale dell'implementazione risiede nella fiducia richiesta ai custodi della chiave per il corretto conteggio dei risultati aggregati. Per risolvere il trade-off tra "Prevenzione della Coercizione" e "Trasparenza Universale", un'estensione futura prevederebbe l'implementazione di ZKP. Ciò consentirebbe all'AE di generare un certificato matematico di corretta decifratura e aggregazione, senza mostrare i singoli voti.
- Decifratura Distribuita Omomorfica: Attualmente, il Threshold Scheme prevede la ricostruzione logica di SK_{AE} in un unico ambiente di memoria durante lo spoglio. Un miglioramento sarebbe far eseguire la decifratura parziale dei ciphertext ai singoli custodi, combinando i frammenti del voto in chiaro senza mai ricostruire o esporre la chiave privata intera sul server dell'AE.

Capitolo 4: Implementazione

In questo ultimo capitolo vengono discusse le scelte di implementazione del sistema e l'analisi delle sue performance rispetto alla complessità temporale e allo spazio di memoria usato.

Link github dell'implementazione :

<https://github.com/Teddyno/PROGETTO-APS-GRUPPO-26---GIUGNO-2026>

4.1 Ambiente Simulato e Scelte Tecnologiche

Il protocollo progettato è stato implementato sviluppando un'applicazione Python. L'applicazione simula su un'unica macchina gli attori principali del sistema (AR, AE, Elettore) misurandone le performance.

4.2 Analisi delle Prestazioni

La simulazione comprende l'intera elezione, dalla Key Distribution alla pubblicazione dei risultati, per un'urna di 100 elettori. I risultati ottenuti tramite il test confermano le stime di alta efficienza esposte nel capitolo 3.3.

I risultati evidenziano l'impatto crittografico per aree di competenza:

- Impatto di rete e dimensione dei messaggi
Il crittogramma della scheda elettorale C_v , generato utilizzando l'algoritmo RSA a 2048-bit, mantiene una dimensione estremamente compatta: 289 bytes (per una scheda con 3 opzioni + 1 nulla). Questo garantisce una comunicazione di rete leggerissima, offrendo una naturale resistenza contro attacchi DDoS.
- Costo computazionale lato elettore
 - Cifratura del voto: l'operazione di cifratura asimmetrica locale, che applica RSA-OAEP con padding probabilistico e l'inserimento del timestamp, richiede 0.05-0.15 millisecondi. Questo conferma che la funzione crittografica vitale del protocollo può essere eseguita in modo impercettibile e fluido su qualsiasi dispositivo.

- Verifica della Merkle Proof: la fase di verificabilità individuale, in cui l'elettore risale l'albero calcolando gli hash SHA-256 dei nodi fratelli, ha una complessità logaritmica $O(\log n)$.

I test mostrano un tempo di esecuzione inferiore ai 0.10 millisecondi, confermando l'eleganza di questa soluzione per garantire la trasparenza individuale abbinata ad un'alta efficienza.

- Costo computazionale lato AE
 - Costruzione del Merkle Tree: l'ordinamento delle foglie e la generazione bottom-up dell'albero, per 100 schede, richiede in media meno di 2 millisecondi.
 - Spoglio e Decifratura: Il processo di mescolamento seguito dalla decifratura RSA sequenziale dell'intera urna richiede in media circa 80 millisecondi (la parte più lenta dell'intero programma).
- Scalabilità: trattandosi di un'operazione crittografica con complessità lineare $O(n)$, lo spoglio scala in modo prevedibile e gestibile. Estrapolando i dati raccolti, la decifratura e l'aggregazione di 1.000.000 di voti richiederebbe circa 10 minuti di elaborazione su un singolo thread.

In conclusione, i risultati confermano che il protocollo è perfettamente applicabile in scenari reali di vasta scala. La scelta di standard come RSA, SHA-256 e Merkle Trees ha permesso di bilanciare in modo ottimale il livello di sicurezza con le prestazioni computazionali.

4.3 Parti mancanti rispetto alla soluzione

Nell'implementazione del sistema, non sono state realizzate alcune parti della soluzione, in particolare:

- La divisione in più della SK_{AE} per la decifrazione dei voti, nel programma è stata solo “simulata” tramite una variabile booleana che indica se la chiave è disponibile (e quindi ricostruita) o meno, a causa dell'impossibilità di dividere la chiave privata in più parti, se non con l'uso di tecniche avanzate e non presenti nel programma del corso. Nel programma la chiave privata non viene mai cancellata dalla memoria e di conseguenza il single point of failure persiste
- La firma sulla SK_{AE} per la decifrazione dei voti da parte del CA, un processo semplice ma lungo da implementare senza apportare effettivamente grandi benefici per il funzionamento della simulazione.